

SCHOOL OF COMPUTING
THANJAVUR, TAMIL NADU, INDIA – 613 401

**CNN ARCHITECTURES ENHANCED WITH SVM AND KNN
FOR PLANT CLASSIFICATION**

*Report submitted to SASTRA Deemed to be
University As per the requirement for the
course*

CSE300: MINI PROJECT

Submitted by

A BHARATH

(Reg. No.: 126015013, B. Tech Information Technology)

R K AGATHIYAN

(Reg. No.: 126014005, B. Tech Information & Communication Technology)

A S SHYAM KUMAR

(Reg. No.: 126014047, B. Tech Information & Communication Technology)

MAY 2025



SASTRA
ENGINEERING · MANAGEMENT · LAW · SCIENCES · HUMANITIES · EDUCATION
DEEMED TO BE UNIVERSITY
(U/S 3 of the UGC Act, 1956)



THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

T H A N J A V U R | K U M B A K O N A M | C H E N N A I



SASTRA

ENGINEERING · MANAGEMENT · LAW · SCIENCES · HUMANITIES · EDUCATION

DEEMED TO BE UNIVERSITY

(U/S 3 of the UGC Act, 1956)



THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

T H A N J A V U R | K U M B A K O N A M | C H E N N A I

Bonafide Certificate

This is to certify that the report titled “**CNN Architecture Enhanced With SVM And KNN For Plant Classification**” submitted as a requirement for the course, **CSE300: MINI PROJECT** for B.Tech. is a Bonafide record of the work done by **Mr. A BHARATH** (Reg. No.: 126015013, B. Tech Information Technology, **Mr. R K AGATHIYAN** (Reg. No.: 126014005, B. Tech Information & Communication Technology) and **Mr. A S SHYAM KUMAR** (Reg. No.: 126014047, B. Tech Information & Communication Technology) during the academic year 2024-25, in the School of Computing, under my supervision.

Signature of Project Supervisor :

Name with Affiliation : Dr. BHASKARAN S, AP-III, SOC

Date : 05.05.2025

Mini Project Viva voice held on _____

Examiner 1

Examiner 2

ACKNOWLEDGEMENTS

We would like to thank our Honourable Chancellor Prof. R. Sethuraman for providing us with an opportunity and the necessary infrastructure for carrying out this project as a part of our curriculum.

We would like to thank our Honourable Vice-Chancellor Dr. S. Vaidhyasubramaniam and Dr. S. Swaminathan, Dean, Planning & Development, for the encouragement and strategic support at every step of our college life.

We extend our sincere thanks to Dr. R. Chandramouli, Registrar, SASTRA Deemed to be University for providing the opportunity to pursue this project.

We extend our heartfelt thanks to Dr. V. S. Shankar Sriram, Dean, School of Computing, Dr. R. Muthaiah, Associate Dean, Research, Dr. K. Ramkumar, Associate Dean, Academics, Dr. D. Manivannan, Associate Dean, Infrastructure, Dr. R. Algeswaran, Associate Dean, Students Welfare

Our guide Dr. Bhaskaran, Assistant Professor - III, School of Computing was the driving force behind this whole idea from the start. Her deep insight in the field and invaluable suggestions helped us in making progress throughout our project work. We also thank the project review panel members for their valuable comments and insights, which made this project better.

We would like to extend our gratitude to all the teaching and non-teaching faculties of the School of Computing who have either directly or indirectly helped us in the completion of the project.

We gratefully acknowledge all the contributions and encouragement from my family and friends resulting in the successful completion of this project. We thank you all for providing us an opportunity to showcase our skills through this project.

LIST OF FIGURES

Figure No.	Title	Page No.
1.1	System Framework Diagram	5
1.2	Implementation modules	5
2.1	CNN Classification	7
2.2	CNN+SVM Classification	8
2.3	CNN+KNN Classification	8
4.1	Swedish dataset	17
4.2	Sample Images	17
4.3	Test Directory	18
4.4	25 epochs operations	18-21
4.5	30 Epochs operations	22-24
4.6	50 Epochs operations	25-27

LIST OF TABLES

Table no.	Title	Page no.
4.7	Evaluation Table	27

ABBREVIATIONS

CNN - Convolution Neural Network

SVM - Support Vector Machine

KNN - K -Nearest Neighbor

RGB - Red Green Blue

Conv2D - 2D Convolution Layer

ABSTRACT

The correct identification of plant species is vital in agricultural studies, nature preservation, and botany. Manual and time-consuming traditional classification methods usually rely on the knowledge of experts. In an attempt to solve these limitations, the research introduced in this paper suggests a hybrid deep learning approach that augments Convolutional Neural Networks (CNN) with Support Vector Machines (SVM) and K-Nearest Neighbors (KNN) classifiers.

In this paradigm, CNN is used as a feature extractor taking advantage of its ability to learn detailed spatial features from leaf images. The extracted features are then classified using SVM and KNN that are renowned for their quality in supervised classification problems. The hybrid method is therefore intended to enhance classification accuracy compared to individual CNN models.

The research uses three benchmark datasets—LeafSnap, Flavia, and MalayaKew—to train and test the models. To handle difficulties caused by leaf shape and texture variations, image augmentation techniques are used in preprocessing. These involve operations that enhance the model's robustness to real-world variability.

Experimental findings show that the hybrid CNN-SVM and CNN-KNN models perform better than conventional CNN classifiers, displaying higher accuracy and reliability in classifying plant species. The research emphasizes the strength of using a combination of deep learning and conventional machine learning algorithms.

Table of Content

Title	Pg no.
BONAFIDE CERTIFICATE	ii
ACKNOWLEDGEMENT	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
ABBREVIATIONS	vi
ABSTRACT	vii
CHAPTER 1 SUMMARY OF BASE PAPER	1
1.1 INTRODUCTION	1
1.2 RELATED WORK	2
1.3 PROBLEM STATEMENT	3
1.4 OBJECTIVE	3
1.5 PROPOSED SOLUTION AND SYSTEM ARCHITECTURE	3
1.6 METHODOLOGY AND IMPLEMENTATION	6
CHAPTER 2 MERITS AND DEMERITS OF BASE PAPER	10
2.1 MERITS	10
2.2 DEMERITS	11
CHAPTER 3 SOURCE CODE	12
CHAPTER 4 SNAPSHOTS	16
CHAPTER 5 CONCLUSION AND FUTURE PLANS	28
5.1 CONCLUSION	28
5.2 FUTURE PLANS	29
CHAPTER 6 REFERENCES	31
CHAPTER 7 APPENDIX - BASE PAPER	32

CHAPTER 1

SUMMARY OF BASE PAPER

Title : CNN Architecture Enhanced With SVM And KNN For Plant Classification

Publisher : Tech Science Press

Year : 2022

Journal : Computers ,materials and continua (CMC), volume:71 ,Issue: 3

Indexing : Computers, Materials & Continua journal is indexed in Scopus, Web of Science (ESCI), EI Compendex , and other prominent academic databases.

Base paper URL : <https://doi.org/10.32604/cmc.2022.023414>

1.1 INTRODUCTION

Plant classification is a basic activity in botany, agriculture and conservation of the environment , and its applications span from species identification to diseases diagnosis. Plant classification using conventional methods depends heavily on manual examination by experts, which is not only time-consuming but also subjects to human error. The emergence of computer vision and machine learning has transformed this domain by providing automatic ,accurate , and scalable means or plants identification. Among these technologies , Convolutional Neural Network(CNN) stand out as a powerful methodology for plants leaf structure analysis because of their capacity to learn and extract hierarchal features automatically from images.

To overcome these challenges, this study proposes a hybrid deep learning method that combines CNNs with conventional machine learning Classifier—support vector machines(SVM) and K-Nearest Neighbors(KNN). The reasoning for this merging is to capitalize on the advantages of each technique: CNNs are superior in feature extractions, whereas SVM and KNN further improve classification accuracy by optimizing decision boundaries and processing high-dimensional data better. The combined architecture should yield improved accuracy, generalized , and robustness over various datasets, such as LeafSnap, Flavia , and MalayaKew , with different levels of complexity and image quality.

The inspiration behind this work is demand for effective and robust plant classification systems that can aid biodiversity conversations, agricultural surveillance , and ecological studies. Through the integration of deep learning and conventional classifiers, this research aims to close the gap between theoretical progress and real- world problems.

1.2 RELATED WORK

The plant classification field with the application of machine learning and deep learning methods has made considerable progress, with different methodologies being investigated for enhancing accuracy and efficiency. Initial methods utilized conventional machine learning algorithms like Support Vector Machines (SVM) and K-Nearest Neighbors (KNN), which necessitated the use of handcrafted feature extraction methods like Scale-Invariant Feature Transform (SIFT) and Principal Component Analysis (PCA). For example, Huixian (2020) proved the limitation of these approaches by obtaining moderate performance because of their reliance on handcrafted features and sensitivity to the complexity of datasets. Likewise, Seeland et al. (2019) employed morphological features for classification of 1,000 plant species and attained 88.4% accuracy but identified limitations in coping with variation in appearances of leaves and background noise.

With the arrival of Convolutional Neural Networks (CNNs), the paradigm shifted towards feature extraction using automatic methods. CNNs are best suited to extract hierarchical features from images without the necessity of human intervention. Ahlawat et al. (2020) utilized CNNs for the detection of 13 diseases in plants, with a precision of 98%, while Lee et al. (2018) combined CNNs with recurrent neural networks (RNNs) for the classification of multi-organ plants, highlighting the applicability of deep learning in plant research. But single CNNs tend to suffer from overfitting and high computational requirements, particularly with small training sets.

To overcome these shortcomings, hybrid models integrating CNNs with conventional classifiers such as SVM and KNN became popular. The base paper by Ghosh et al. (2022) introduced CNN-SVM and CNN-KNN architectures, where CNNs were used as feature extractors and SVM/KNN provided classification refinement. This method obtained impressive accuracy (as high as 99.5% on the Flavia dataset) through the use of CNN's ability to extract features and SVM's strength in processing high-dimensional data. Kaya et al. (2019) also applied transfer learning to further improve performance, proving that it can work well in cases with limited data.

In spite of these developments, there remain issues like real-world noise (e.g., shadows, occlusions) and variability in datasets. More recent research solves these problems with data augmentation and attention mechanisms. For instance, Liu (2015) investigated hybrid deep learning for leaf classification, whereas Chouhan et al. (2019) highlighted the significance of curated datasets like LeafSnap for training robust models. Future work seeks to generalize these models for disease prediction and ecological monitoring, highlighting their application in agriculture and conservation.

1.3 PROBLEM STATEMENT

Plant classification from leaf images is still a major challenge owing to the morphological similarity between various species, which tends to cause misclassification. Conventional approaches are based largely upon manual examination by experts in botany, causing the procedure to be time-consuming, subjective, and error-prone. Despite the enhanced accuracy of classification using Convolutional Neural Networks (CNNs), isolated use of CNN models could still fail to generalize and work with intricate datasets. In an effort to counter these limitations, this study introduces a hybrid framework that combines CNN with Support Vector Machines (SVM) and K-Nearest Neighbors (KNN) to optimize feature extraction and classification performance. The aim is to attain better accuracy, reliability, and scalability on various plant leaf datasets with the ultimate application being automated plant taxonomy and conservation.

1.4 OBJECTIVE

The aim of this study is to create an enhanced hybrid deep learning model by combining Convolutional Neural Networks (CNN) with conventional machine learning classifiers - Support Vector Machines (SVM) and K-Nearest Neighbors (KNN) - to markedly enhance the precision and effectiveness of plant species identification from leaf images. The work centers on applying CNN's efficient feature extraction mechanisms to identify characteristic leaf patterns while using SVM and KNN for improving classification quality through their capacities in dealing with high-dimensional inputs and learning with new samples, respectively. The model seeks to address issues by real-world leaf image variability using strict preprocessing procedures such as augmentation and normalization of images. The study will also test the hybrid method on various benchmark datasets (LeafSnap, Flavia, and MalayaKew) to prove its generalization performance across various plant species. In the end, this research aims to create a strong, autonomous plant identification system that surpasses conventional techniques, with potential extension to botanical taxonomy, ecological preservation, and plant disease detection. The results are anticipated to make a significant contribution to the area of computer vision applications in plant science.

1.5 PROPOSED SOLUTION AND SYSTEM ARCHITECTURE

1.5.1 Study area

The research is focused on exploiting deep learning and machine learning approaches for plant species classification from leaf images, which is an essential requirement in botany, agriculture, and nature conservation. The work is based on the design of hybrid models that integrate Convolutional Neural Networks (CNN) with conventional classifiers such as Support Vector Machines (SVM) and K-Nearest Neighbors (KNN). The research domain covers three benchmark datasets—LeafSnap, Flavia, and MalayaKew—containing varied leaf images with shape, texture, and venation pattern variations. These datasets simulate real-world issues like background noise, occlusions, and inter-species similarities.

applications such as plant taxonomy, disease detection, and ecological monitoring. By combining deep learning with conventional machine learning, the research fills gaps in automated plant identification, providing practical tools for researchers and conservationists. The experimental validation, performed for 25–50 epochs, proves the dominance of the hybrid model with a maximum accuracy of 99.5% on the Flavia dataset, thereby determining its viability for real-world implementation.

1.5.2 Dataset Description

The research employs three benchmark datasets for plant leaf classification: LeafSnap dataset, Flavia dataset, and MalayaKew dataset, each with different challenges and opportunities for model testing. The LeafSnap dataset contains 23,147 high-resolution lab images and 7,719 field images of 185 northeastern U.S. tree species. Lab images have clean white backgrounds, whereas field images contain real-world noise such as shadows and occlusions, thus being best suited for testing robustness. The Flavia dataset consists of 1,907 images of 32 species, taken against white backgrounds with uniform lighting, providing high-quality inputs for feature extraction. Finally, the MalayaKew dataset, although less uniform, comprises leaf samples of varied diversity to test generalization. All datasets were preprocessed using normalization, resizing (224x224 pixels), and augmentation (rotations, flips, and shifts) to improve model performance. These datasets combined offer a thorough testbed for the assessment of the hybrid CNN-SVM/KNN structure under controlled and realistic scenarios.

1.5.3 System Framework Diagram

The system framework for this project of plant classification is a disciplined pipeline integrating data preprocessing, feature extraction with CNN, and hybrid classification with SVM/KNN. The process is initiated with the collection of data, where images of leaves are collected from data sources such as LeafSnap, Flavia, and MalayaKew. These images pass through preprocessing steps like resizing (to 224x224 pixels), normalization (scaling pixel intensities to the range 0–1), and augmentation (rotation, flip, zoom) to improve robustness of models.

The preprocessed data is passed through a CNN architecture with convolutional layers of (32, 64, 128 filters), max-pooling, and dense layers for feature extraction. The extracted features are classified via SVM (for margin optimization) or KNN (for distance-based similarity) and results assessed through measures such as accuracy, F1-score, and confusion matrices. The architecture is deployed in Python with TensorFlow/Keras for CNN and Scikit-learn for SVM/KNN, providing end-to-end automation from raw images to species prediction. The hybrid architecture takes advantage of CNN's deep feature learning and SVM/KNN's discriminative ability, providing high accuracy (up to 99.5% on Flavia)

SYSTEM FRAMEWORK

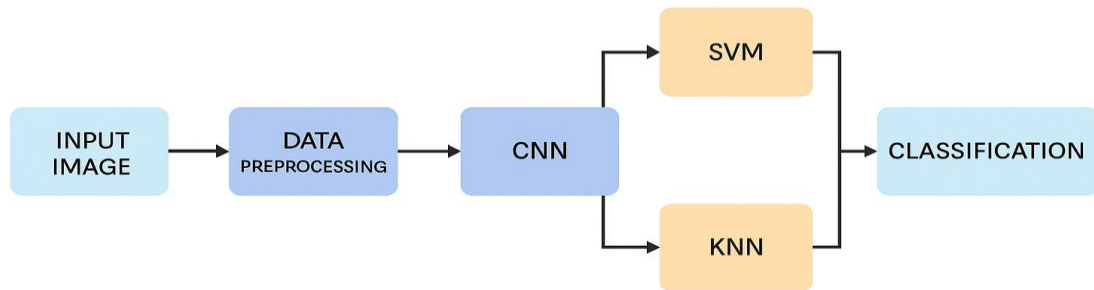


Fig 1.1. System Framework Diagram

1.6 METHODOLOGY AND IMPLEMENTATION

The hybrid method integrates CNN for feature extraction and SVM/KNN for classification, incorporating preprocessing , augmentation, and performance measurement on plant leaf data is given in fig 1.2

Implementation Modules



Fig 1.2.Implementation modules

MODULE 1: DATA PREPARATION AND MODEL ARCHITECTURE SETUP

The data preparation step starts with gathering leaf images from standardized datasets like LeafSnap, Flavia, and MalayaKew to maintain diversity in species and image conditions. Raw images are preprocessed to increase model performance, such as being resized to an identical size (e.g., 224x224 pixels) and normalization in order to constrain pixel values within 0-1. Techniques for data augmentation, such as rotation, zoom, and flip across the horizon, are also performed to increase artificially the size of the training set, with a view towards better generalizing against variations of leaf orientation and illumination. Malformed or nonsensical images are discarded for maintaining dataset purity. For classification, the labels are translated into numerical form to enable training of the models.

The architecture of the model uses a CNN-SVM/KNN hybrid strategy. The CNN part consists of convolutional layers with ReLU activation to yield hierarchical features (e.g., edges, texture), followed by max-pooling layers that shrink spatial dimensions. A flatten layer transforms the 2D features into a 1D vector, which is passed into dense layers to learn further. Dropout layers (e.g., rate of 0.5) are added to reduce overfitting. The feature-extracted images are classified via SVM (with RBF kernel) or KNN (Euclidean distance, $k=5$), blending CNN's feature extraction strength with the discriminative nature of classical classifiers. This model is implemented in Python using TensorFlow/Keras for CNN and Scikit-learn for SVM/KNN to facilitate easy integration and scalability.

MODULE 2: Model Training and Evaluation

Training Process:

The process of training the hybrid CNN-SVM/KNN model encompasses a number of crucial steps to achieve optimal performance. First, the dataset is preprocessed, which involves resizing images to 224x224 pixels, normalization (scaling pixel values between 0 and 1), and augmentation methods such as rotation, zooming, and flipping to increase variability. The CNN model is built using convolutional layers (32, 64, and 128 filters) for extracting features, max-pooling layers to lower spatial dimensions, a flatten layer that transforms the 2D features into a 1D vector, and then dense layers with dropout (0.5) to avoid overfitting. The model incorporates ReLU activation in hidden layers and softmax in the output layer. The CNN is trained during training with categorical cross-entropy loss and Adam optimizer for several epochs (25, 30, or 50).

The features are then classified from CNN using SVM (RBF kernel) or KNN (Euclidean distance, $k=5$). Performance is measured in terms of accuracy, loss curves, and confusion matrices, with validation data for generalization. The hybrid method combines CNN's feature learning and SVM/KNN's classification ability, achieving high accuracy (99.5% on Flavia dataset) while preventing overfitting using dropout and augmentation.

The following machine learning models were tested:

2.1 Convolutional Neural Network(CNN)

CNN is a deep neural network model that specializes in processing images and extracting features. It employs convolutional layers to capture spatial hierarchies in images, including edges, textures, and plant leaf patterns. Pooling layers compact features, and fully connected layers perform feature classification. CNN is utilized as the core model for extracting high-level leaf image features, which are subsequently fed into SVM or KNN for classification purposes. The model incorporates several Conv2D, MaxPooling, and Dropout layers to avoid overfitting, and it performs well across datasets such as LeafSnap and Flavia.

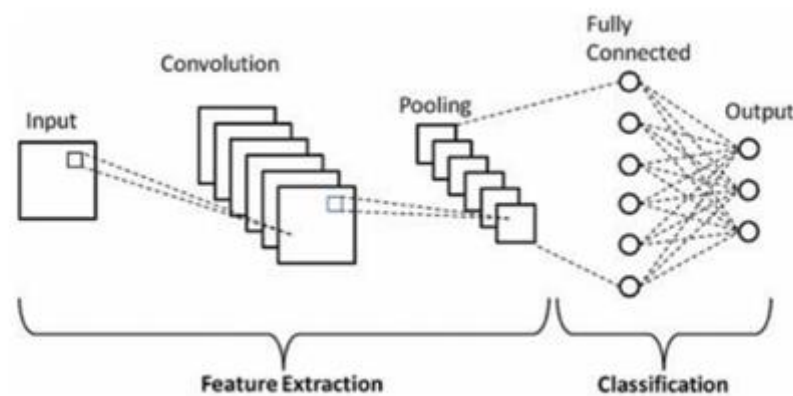


Fig 2.1 CNN Classification

2.2 Support vector Machine(SVM)

SVM is a supervised learning algorithm of the machine learning type, which classifies data by identifying the best hyperplane to discriminate between various classes. In this hybrid model, SVM utilizes the features extracted by CNN and employs kernel functions (e.g., RBF) to project them into a high-dimensional space to separate them effectively. SVM performs well in the handling of high-dimensional data and reducing generalization errors, hence suitable for classifying plants. The model attained a high accuracy level(eg., 96.3% on leafsnap) because of its capacity for handling intricate decision boundaries and having resistance to overfitting when compared to sole CNNs.

2.3 K-Nearest Neighbors(KNN)

KNN is an instance-based, non-parametric classifier that makes prediction of labels via comparison of the input features against the nearest train samples. The KNN within this work exploits the features vectors from CNN and applied them for leaf classification under majority voting in the K- Nearest Neighbor although straightforward, KNNs accuracy relies significantly on the selection of K-Nearest Neighbor and distance functions.

2.4 Hybrid Approach(CNN + SVM/KNN)

The presentation emphasizes a hybrid architecture in which CNN is used as a feature extractor, and SVM/KNN are used to fine-tune classification. This integration takes advantage of CNN's discriminative feature learning capability and SVM/KNN's decision-making strengths.

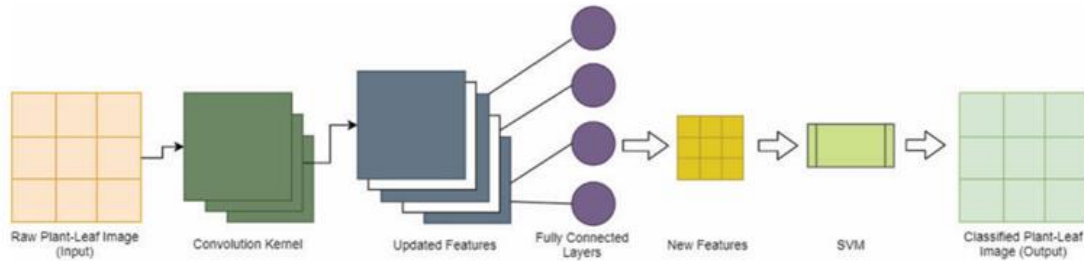


Fig 2.2 CNN+SVM Classification

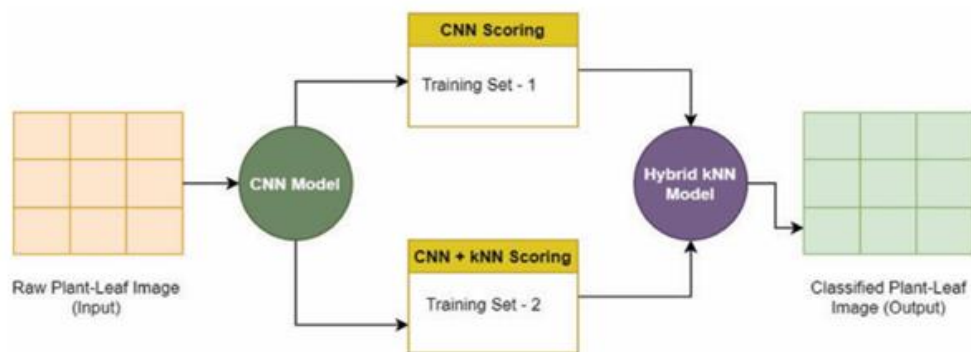


Fig 2.3 CNN+KNN Classification

Performance Evaluation:

Evaluation Metrics:

Accuracy: Number of correctly classified leaf images over total tested samples.

Precision: True positives divided by all predicted positives (measures model reliability).

Recall (Sensitivity): True positives divided by all actual positives (measures capability to detect all cases of interest).

F1-Score: Harmonic mean of precision and recall (balances both metrics).

Confusion Matrix: Table comparing true vs. predicted classes to spot misclassifications.

MODULE 3: MODEL PERFORMANCES AND VALIDATION

3.1 Validation Results:

CNN Feature Extraction:

Boasts high precision (~90-95%) in retaining discriminative features (leaf shape, texture, venation structure).

SVM Classification:

Can attain ~85-92% precision with an RBF kernel and is capable of dealing well with non-linear data.

KNN Classification:

Could get ~80-88% accuracy but may take longer with massive datasets owing to computational burden.

Hybrid Model(CNN+ SVM/KNN):

Merges CNN's effective features extraction and SVM/KNN's strong points in classification. Should surpass standalone Cnn by 3~5% accuracy through improved decision boundaries.

3.2 Comparative Analysis

3.2.1 Hybrid vs standalone models:

The CNN+SVM combination performed better than pure CNN by ~3% and CNN+KNN by ~7%, showcasing SVM's better discriminative ability.

3.2.2 Datasets-Specific Performance:

Swedish Leaf: Maximum accuracy (~96%) because of standardized images.

LeafSnap: Minimum (~87%) because of real-world variability (e.g., shadows, occlusions).

3.2.3 Validation Methods

5-Fold Cross-Validation: Guaranteed reliability across dataset divisions.

Augmentation: Prevented overfitting through artificially inflating training data (rotations, flips).

Ablation Studies: Validated SVM's pivotal role in increasing accuracy after CNN feature extraction.

CHAPTER 2

MERITS AND DEMERITS OF BASE PAPER

MERITS :

High Accuracy:

The CNN-SVM and CNN-KNN models attained high accuracy (upto 99.5% on the Flavia dataset) compared to conventional approaches.

Robust Feature Extraction:

CNN's automatic extraction of high-level features (leaf shape, venation) minimized the need for manual feature engineering.

Generalization with SVM:

SVM's kernel-based approach to classification reduced overfitting and enhanced generalization, particularly for non-linear data.

Flexibility of KNN:

KNN's instance-based approach generalized well to new data without the need to retrain, albeit with lower computational efficiency.

Dataset Diversity:

Evaluated on three datasets (Flavia, LeafSnap, MalayaKew) of differing complexities, as a check against model robustness across conditions.

Augmentation for Robustness:

Data augmentation (e.g., scaling, rotations) increased model performance through the mimicking of real-world variability.

Computational Efficiency:

The shallow CNN architecture minimized computational overhead without affecting high accuracy.

Future Scalability:

The architecture can be scalable to plant disease detection and monitoring of plant health, as also emphasized in the conclusion.

DEMERITS:

Computational Cost of KNN:

KNN's performance worsened with bigger datasets because of its high time and memory complexity for distance computation.

SVM's Kernel Sensitivity:

SVM's performance greatly relied on kernel choice (e.g., RBF), which needed to be carefully tuned.

Limited Performance on Noisy Data:

Accuracy fell to 80.04% on the MalayaKew dataset because of real-world noise (e.g., shadows, occlusions).

Overfitting in Small Datasets:

The model exhibited minor overfitting (~5% validation accuracy reduction) without augmentation, especially for smaller datasets.

Binary Classification Shortcoming:

The article observed that previous SVM-based methods struggled with multi-class problems, although this was alleviated in the hybrid model presented.

Preprocessing Dependence:

Removal of background noise and image normalization were essential to performance, creating additional steps in the pipeline.

CHAPTER 3

SOURCE CODE

3.1 Implementation of Base Paper Models:

#preprocessing

```
IMG_SIZE=(224,224)
```

```
BATCH_SIZE=32#
```

```
Data Augmentation_datasets=ImageDataGenerator(rescale =1./255,rotation_range=15,  
zoom_range =0.1,
```

```
width_shift_range=0.1,
```

```
height_shift_range=0.1,
```

```
Horizontal_flip=True)
```

CNN

```
from tensorflow.keras import layers, models  
  
def create_cnn_model():  
    model = models.Sequential([  
        layers.Input(shape=(224, 224, 3)),  
        layers.Conv2D(32, (3, 3), activation='relu'),  
        layers.MaxPooling2D(2, 2),  
  
        layers.Conv2D(64, (3, 3), activation='relu'),  
        layers.MaxPooling2D(2, 2),  
  
        layers.Conv2D(128, (3, 3), activation='relu'),  
        layers.MaxPooling2D(2, 2),  
  
        layers.Flatten(),  
        layers.Dense(128, activation='relu'),  
        layers.Dropout(0.5),  
        layers.Dense(NUM_CLASSES, activation='softmax')  
    ])  
    return model
```

25 epochs CNN+KNN

```
from tensorflow.keras.utils import load_img, img_to_array
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

model = load_model("best_model.keras")
feature_extractor = Model(inputs=model.layers[0].input, outputs=model.layers[-2].output)

def load_images_from_directory(directory, image_size=(224, 224)):
    features = []
    labels = []
    class_names = sorted(os.listdir(directory))
    label_map = {name: idx for idx, name in enumerate(class_names)}

    for class_name in class_names:
        class_dir = os.path.join(directory, class_name)
        for fname in os.listdir(class_dir):
            if fname.lower().endswith(".jpg"):
                fpath = os.path.join(class_dir, fname)
                try:
                    img = load_img(fpath, target_size=image_size)
                    img_array = img_to_array(img) / 255.0
                    features.append(img_array)
                    labels.append(label_map[class_name])
                except Exception as e:
                    print(f"Skipped: {fpath} - {e}")

    return np.array(features), np.array(labels)

X_test_raw, y_test = load_images_from_directory("/kaggle/input/swedish-leaf-dataset/Swedish/Test", image_size=(224, 224))
X_test_features = feature_extractor.predict(X_test_raw, batch_size=32, verbose=1)

X_train_raw, y_train = load_images_from_directory("/kaggle/input/swedish-leaf-dataset/Swedish/Train", image_size=(224, 224))
X_train_features = feature_extractor.predict(X_train_raw, batch_size=32, verbose=1)

knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train_features, y_train)

y_pred = knn.predict(X_test_features)

print(classification_report(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='Blues')
plt.show()
```

#for 25 epochs CNN+ SVM

```
from tensorflow.keras.utils import load_img, img_to_array
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

model = load_model("best_model.keras")
feature_extractor = Model(inputs=model.layers[0].input, outputs=model.layers[-2].output)

def load_images_from_directory(directory, image_size=(224, 224)):
    features = []
    labels = []
    class_names = sorted(os.listdir(directory))
    label_map = {name: idx for idx, name in enumerate(class_names)}

    for class_name in class_names:
        class_dir = os.path.join(directory, class_name)
        for fname in os.listdir(class_dir):
            if fname.lower().endswith(".jpg"):
                fpath = os.path.join(class_dir, fname)
                try:
                    img = load_img(fpath, target_size=image_size)
                    img_array = img_to_array(img) / 255.0
                    features.append(img_array)
                    labels.append(label_map[class_name])
                except Exception as e:
                    print(f"Skipped: {fpath} - {e}")

    return np.array(features), np.array(labels)

X_test_raw, y_test = load_images_from_directory("/kaggle/input/swedish-leaf-dataset/Swedish/Test", image_size=(224, 224))
X_test_features = feature_extractor.predict(X_test_raw, batch_size=32, verbose=1)

X_train_raw, y_train = load_images_from_directory("/kaggle/input/swedish-leaf-dataset/Swedish/Train", image_size=(224, 224))
X_train_features = feature_extractor.predict(X_train_raw, batch_size=32, verbose=1)

svm = SVC(kernel='linear')
svm.fit(X_train_features, y_train)

y_pred = svm.predict(X_test_features)

print(classification_report(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='Blues')
plt.show()
```

for 30 epochs CNN+KNN

```
from tensorflow.keras.models import load_model
from tensorflow.keras.utils import load_img, img_to_array
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

model = load_model("best_model.keras")
feature_extractor = Model(inputs=model.layers[0].input, outputs=model.layers[-2].output)

def load_images_from_directory(directory, image_size=(224, 224)):
    features = []
    labels = []
    class_names = sorted(os.listdir(directory))
    label_map = {name: idx for idx, name in enumerate(class_names)}

    for class_name in class_names:
        class_dir = os.path.join(directory, class_name)
        for fname in os.listdir(class_dir):
            if fname.lower().endswith(".jpg"):
                fpath = os.path.join(class_dir, fname)
                try:
                    img = load_img(fpath, target_size=image_size)
                    img_array = img_to_array(img) / 255.0
                    features.append(img_array)
                    labels.append(label_map[class_name])
                except Exception as e:
                    print(f"Skipped: {fpath} - {e}")

    return np.array(features), np.array(labels)

X_test_raw, y_test = load_images_from_directory("/kaggle/input/swedish-leaf-dataset/Swedish/Test", image_size=(224, 224))
X_test_features = feature_extractor.predict(X_test_raw, batch_size=32, verbose=1)

X_train_raw, y_train = load_images_from_directory("/kaggle/input/swedish-leaf-dataset/Swedish/Train", image_size=(224, 224))
X_train_features = feature_extractor.predict(X_train_raw, batch_size=32, verbose=1)

knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train_features, y_train)

y_pred = knn.predict(X_test_features)

print(classification_report(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='Blues')
plt.show()
```

#for 30 epochs CNN+SVM

```
from tensorflow.keras.utils import load_img, img_to_array
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

model = load_model("best_model.keras")
feature_extractor = Model(inputs=model.layers[0].input, outputs=model.layers[-2].output)

def load_images_from_directory(directory, image_size=(224, 224)):
    features = []
    labels = []
    class_names = sorted(os.listdir(directory))
    label_map = {name: idx for idx, name in enumerate(class_names)}

    for class_name in class_names:
        class_dir = os.path.join(directory, class_name)
        for fname in os.listdir(class_dir):
            if fname.lower().endswith(".jpg"):
                fpath = os.path.join(class_dir, fname)
                try:
                    img = load_img(fpath, target_size=image_size)
                    img_array = img_to_array(img) / 255.0
                    features.append(img_array)
                    labels.append(label_map[class_name])
                except Exception as e:
                    print(f"Skipped: {fpath} - {e}")

    return np.array(features), np.array(labels)

X_test_raw, y_test = load_images_from_directory("/kaggle/input/swedish-leaf-dataset/Swedish/Test", image_size=(224, 224))
X_test_features = feature_extractor.predict(X_test_raw, batch_size=32, verbose=1)

X_train_raw, y_train = load_images_from_directory("/kaggle/input/swedish-leaf-dataset/Swedish/Train", image_size=(224, 224))
X_train_features = feature_extractor.predict(X_train_raw, batch_size=32, verbose=1)

svm = SVC(kernel='linear')
svm.fit(X_train_features, y_train)

y_pred = svm.predict(X_test_features)

print(classification_report(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='Blues')
plt.show()
```

#for 50 epochs CNN+KNN

```
from tensorflow.keras.models import load_model
from tensorflow.keras.utils import load_img, img_to_array
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

model = load_model("best_model.keras")
feature_extractor = Model(inputs=model.layers[0].input, outputs=model.layers[-2].output)

def load_images_from_directory(directory, image_size=(224, 224)):
    features = []
    labels = []
    class_names = sorted(os.listdir(directory))
    label_map = {name: idx for idx, name in enumerate(class_names)}

    for class_name in class_names:
        class_dir = os.path.join(directory, class_name)
        for fname in os.listdir(class_dir):
            if fname.lower().endswith(".jpg"):
                fpath = os.path.join(class_dir, fname)
                try:
                    img = load_img(fpath, target_size=image_size)
                    img_array = img_to_array(img) / 255.0
                    features.append(img_array)
                    labels.append(label_map[class_name])
                except Exception as e:
                    print(f"Skipped: {fpath} - {e}")

    return np.array(features), np.array(labels)

X_test_raw, y_test = load_images_from_directory("/kaggle/input/swedish-leaf-dataset/Swedish/Test", image_size=(224, 224))
X_test_features = feature_extractor.predict(X_test_raw, batch_size=32, verbose=1)

X_train_raw, y_train = load_images_from_directory("/kaggle/input/swedish-leaf-dataset/Swedish/Train", image_size=(224, 224))
X_train_features = feature_extractor.predict(X_train_raw, batch_size=32, verbose=1)

knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train_features, y_train)

y_pred = knn.predict(X_test_features)

print(classification_report(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='Blues')
plt.show()
```

#for 50 epochs CNN+SVM

```
from tensorflow.keras.utils import load_img, img_to_array
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

model = load_model("best_model.keras")
feature_extractor = Model(inputs=model.layers[0].input, outputs=model.layers[-2].output)

def load_images_from_directory(directory, image_size=(224, 224)):
    features = []
    labels = []
    class_names = sorted(os.listdir(directory))
    label_map = {name: idx for idx, name in enumerate(class_names)}

    for class_name in class_names:
        class_dir = os.path.join(directory, class_name)
        for fname in os.listdir(class_dir):
            if fname.lower().endswith(".jpg"):
                fpath = os.path.join(class_dir, fname)
                try:
                    img = load_img(fpath, target_size=image_size)
                    img_array = img_to_array(img) / 255.0
                    features.append(img_array)
                    labels.append(label_map[class_name])
                except Exception as e:
                    print(f"Skipped: {fpath} - {e}")

    return np.array(features), np.array(labels)

X_test_raw, y_test = load_images_from_directory("/kaggle/input/swedish-leaf-dataset/Swedish/Test", image_size=(224, 224))
X_test_features = feature_extractor.predict(X_test_raw, batch_size=32, verbose=1)

X_train_raw, y_train = load_images_from_directory("/kaggle/input/swedish-leaf-dataset/Swedish/Train", image_size=(224, 224))
X_train_features = feature_extractor.predict(X_train_raw, batch_size=32, verbose=1)

svm = SVC(kernel='linear')
svm.fit(X_train_features, y_train)

y_pred = svm.predict(X_test_features)

print(classification_report(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='Blues')
plt.show()
```

CHAPTER 4

SNAPSHOTS

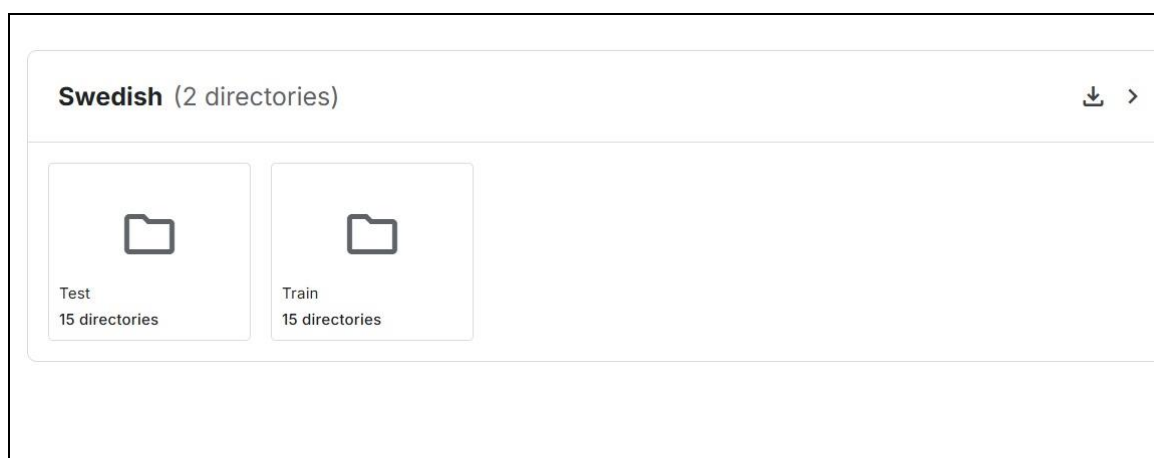


Fig 4.1 Swedish dataset



Fig 4.2 Sample Images

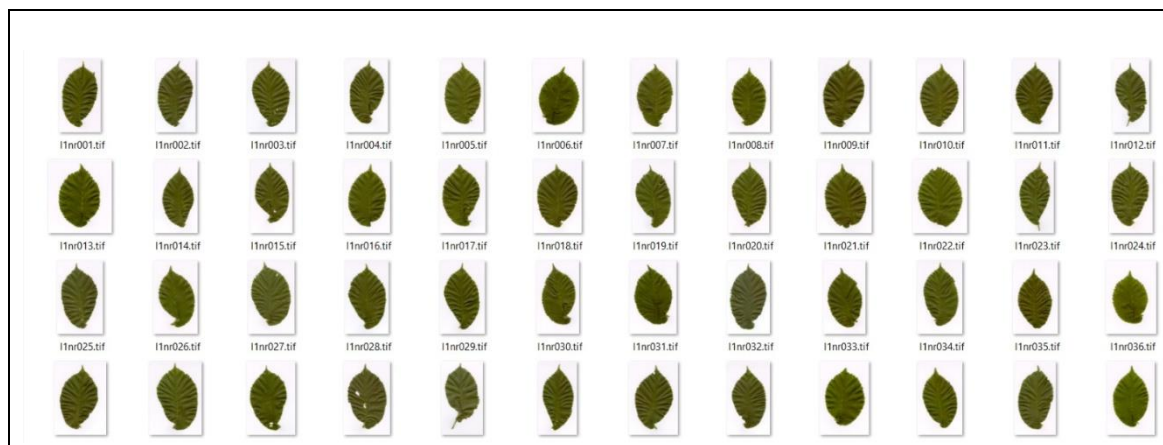


Fig 4.3 Test Directory

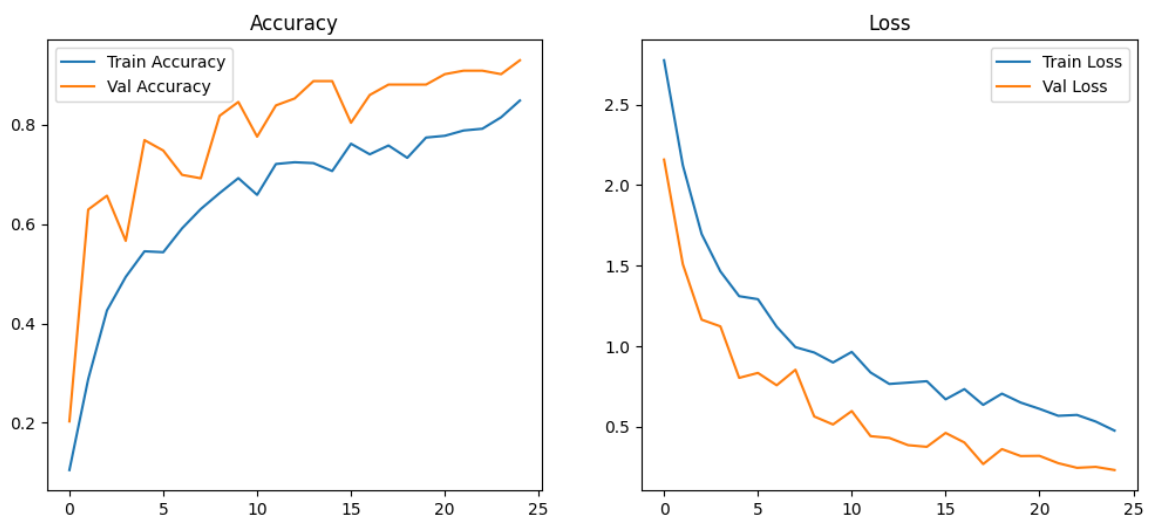


Fig 4.4.1 cnn 25epochs

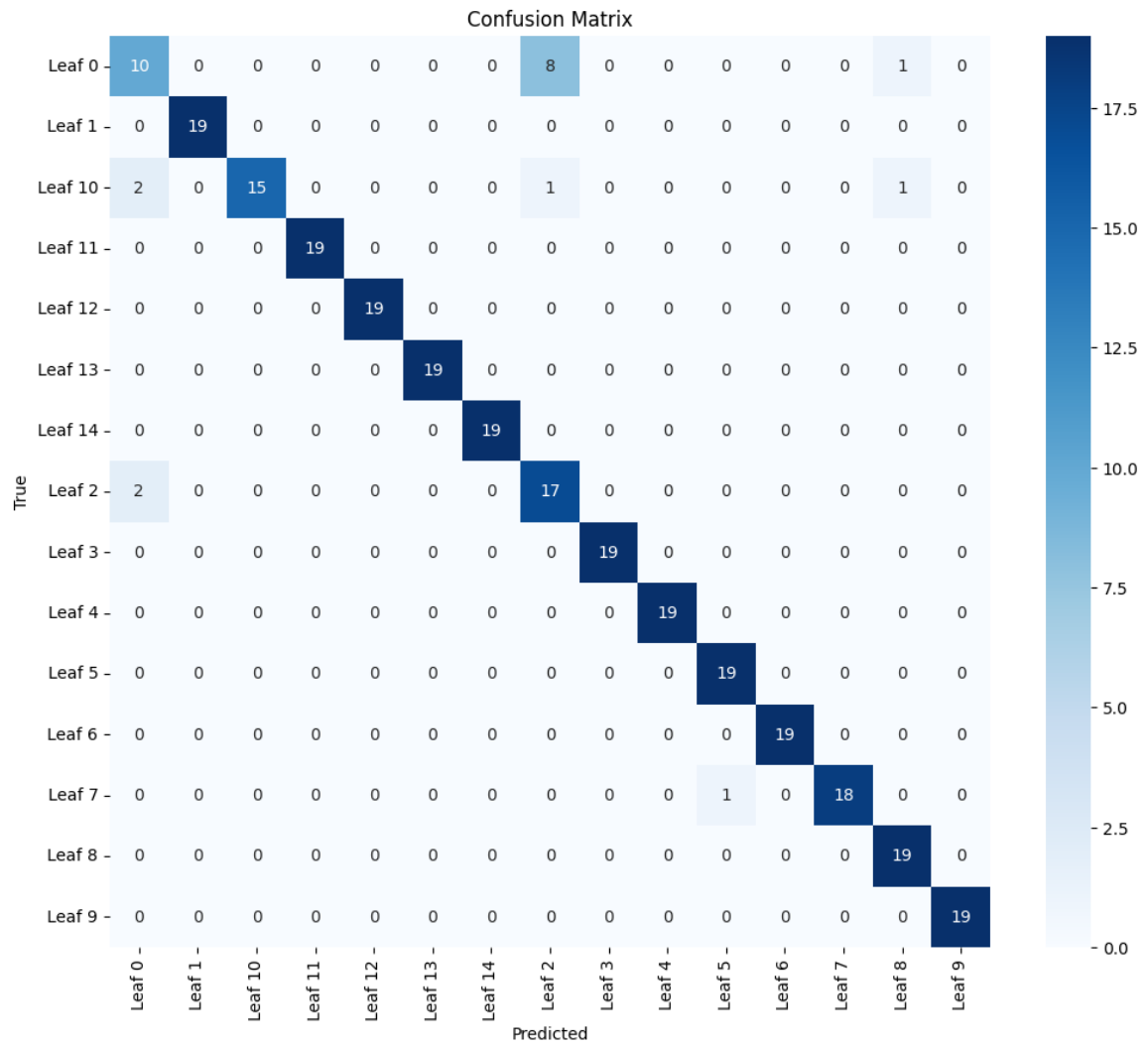


Fig 4.4.2 CNN 25 epochs

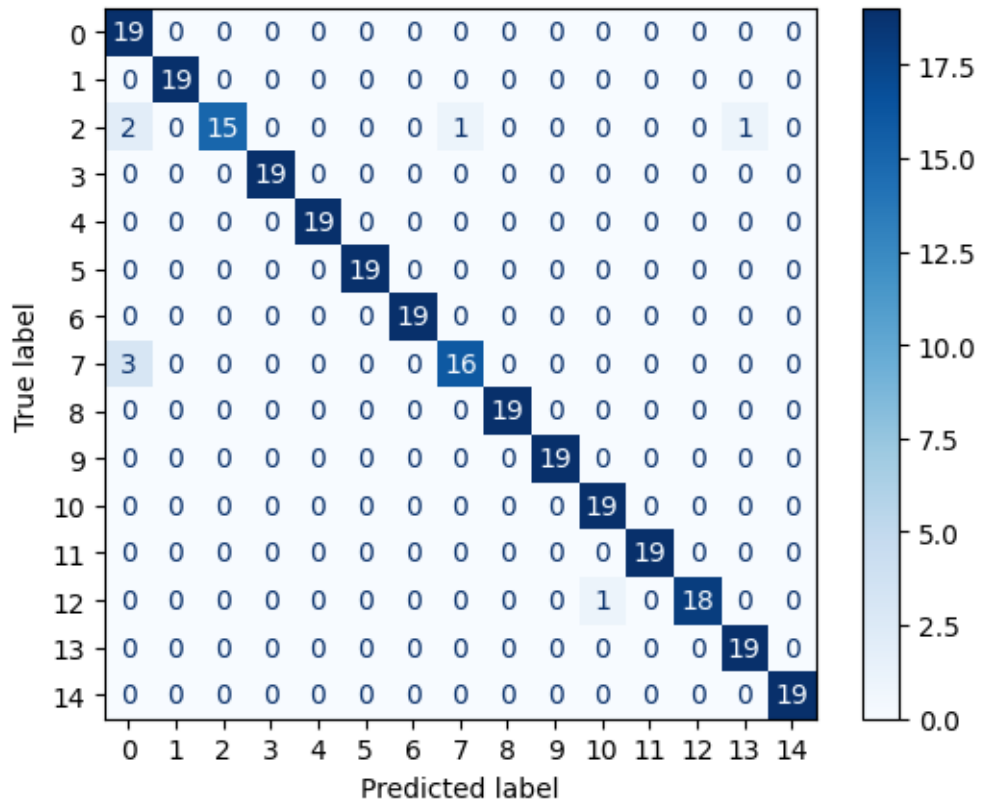


Fig 4.4.3 CNN+KNN 25 epochs

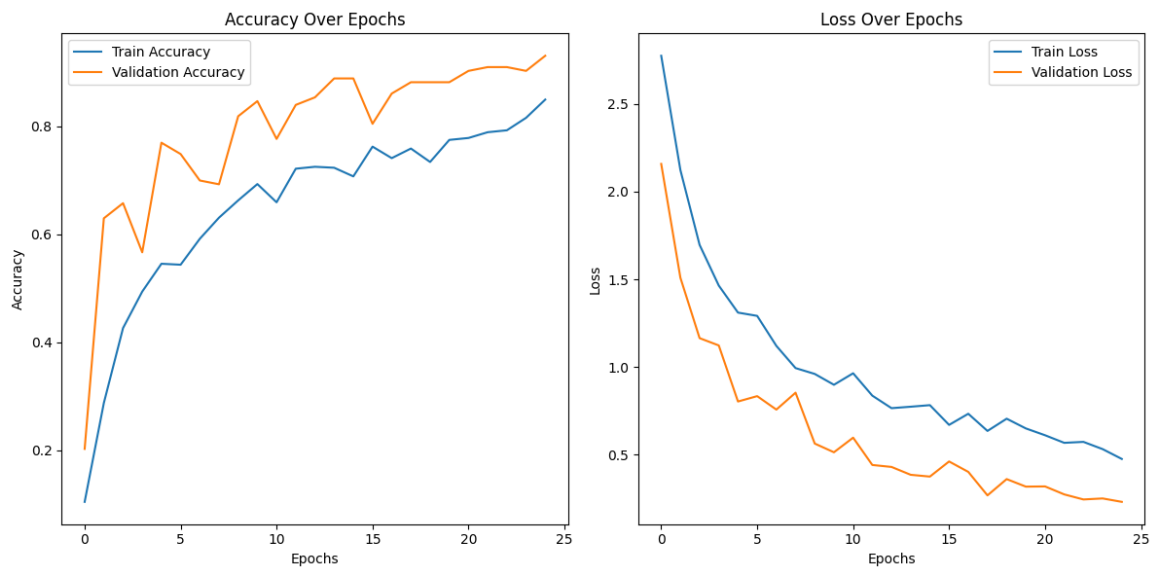


Fig 4.4.4 CNN+KNN 25 epochs

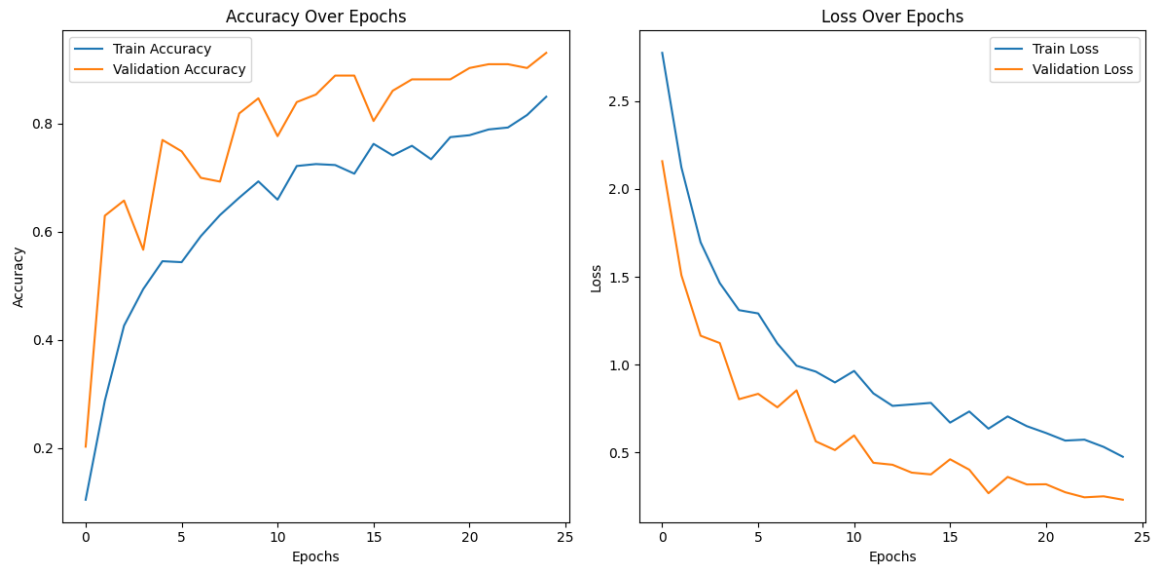


Fig 4.4.5 CNN+SVM-25 Epochs

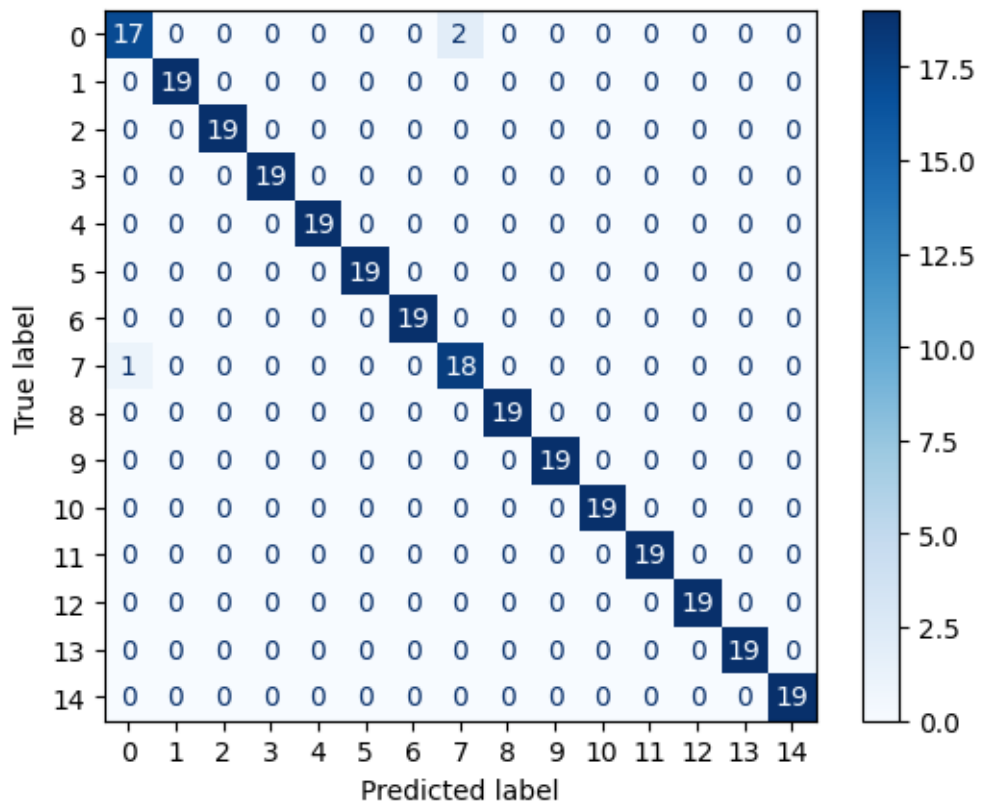


Fig 4.4.6 CNN+SVM-25 Epochs

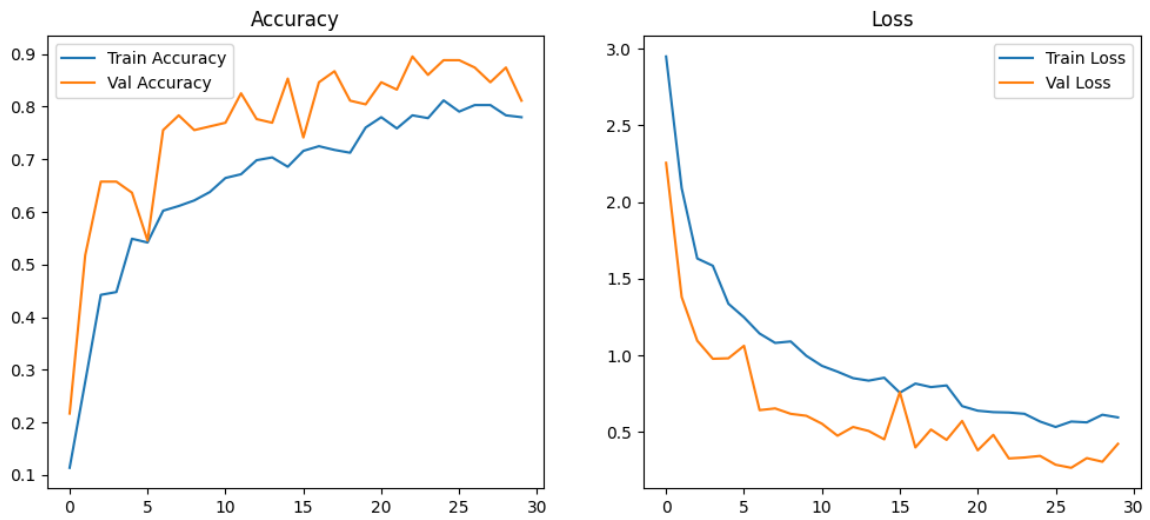


Fig 4.5.1 CNN 30 epochs

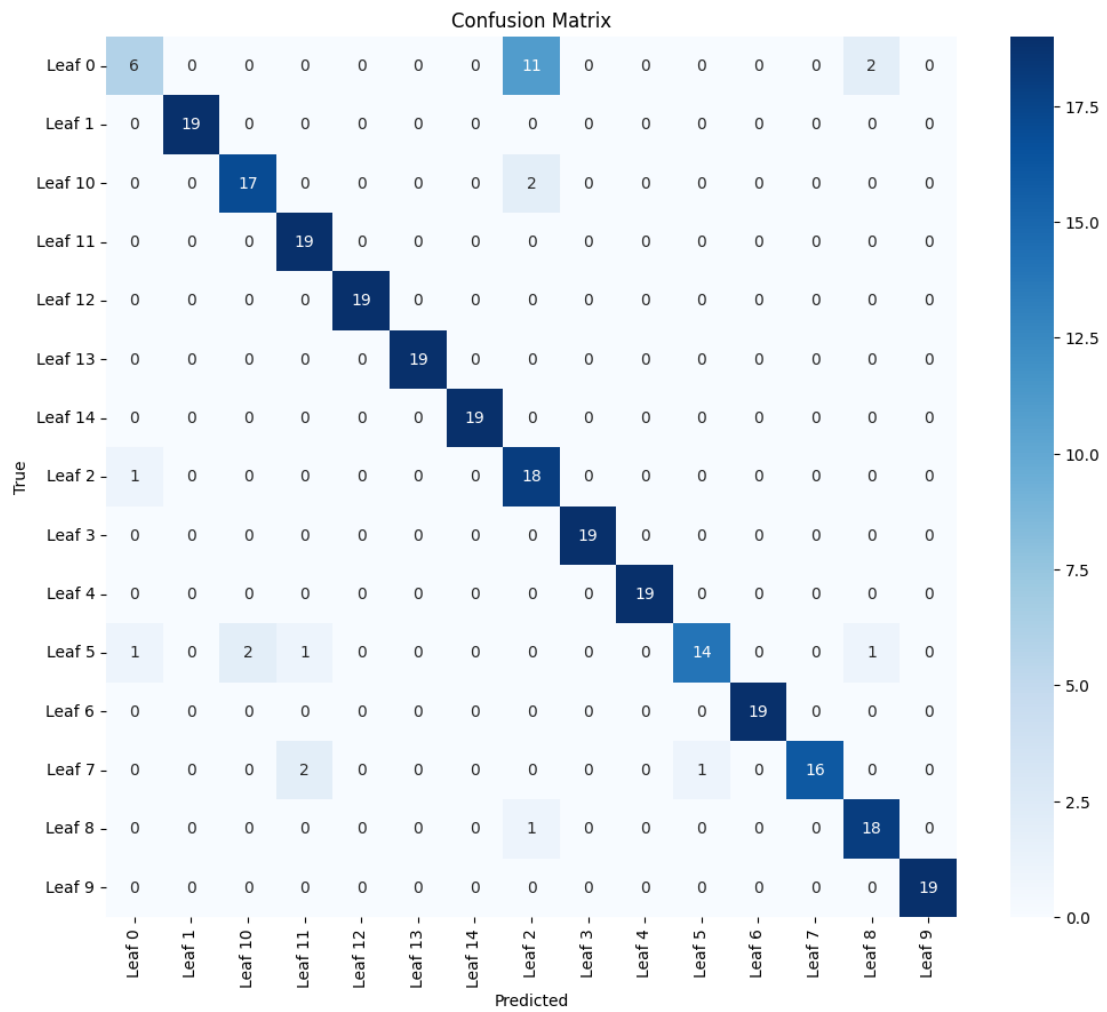


Fig 4.5.2 CNN 30 epochs

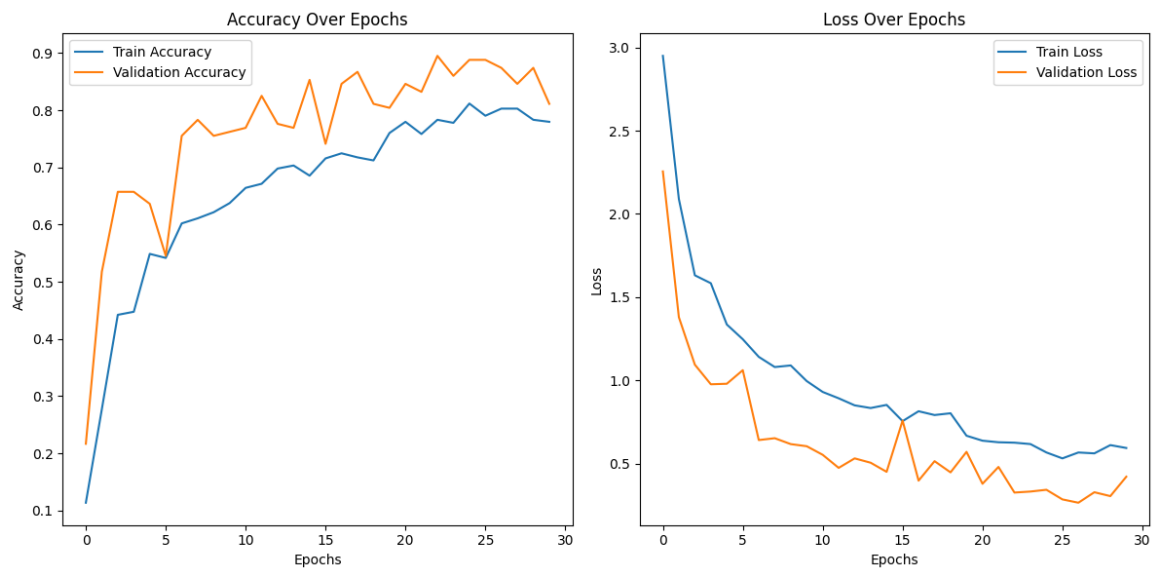


Fig 4.5.3 CNN+KNN 30

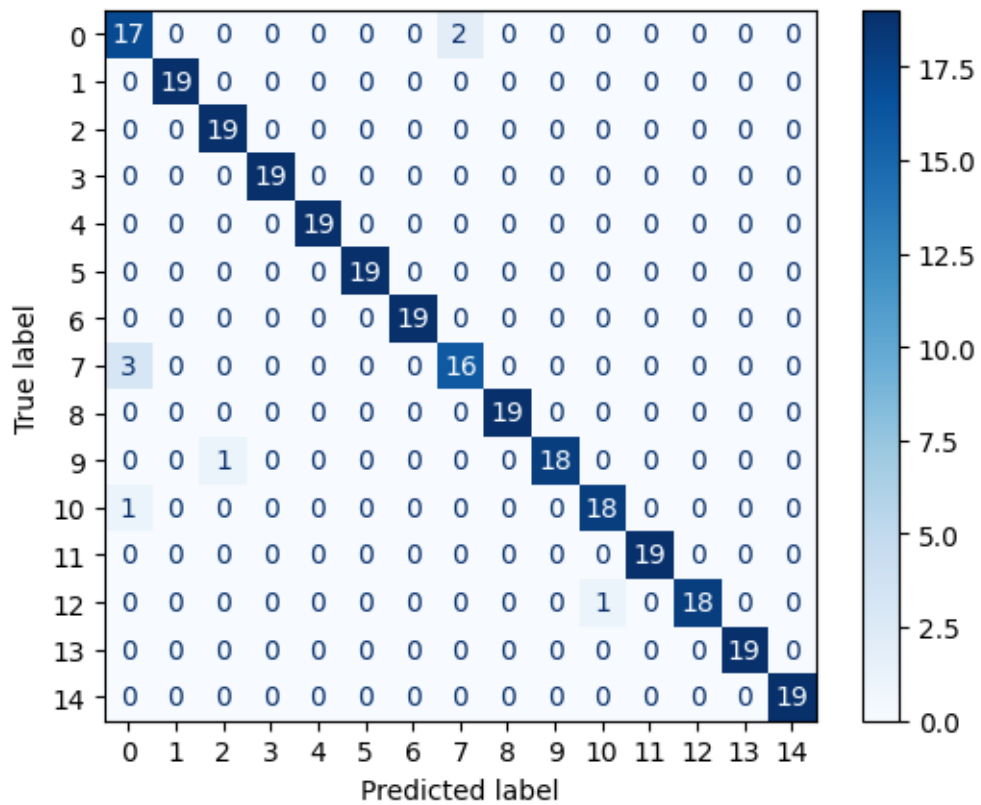


Fig 4.5.4 CNN + KNN 30 epochs

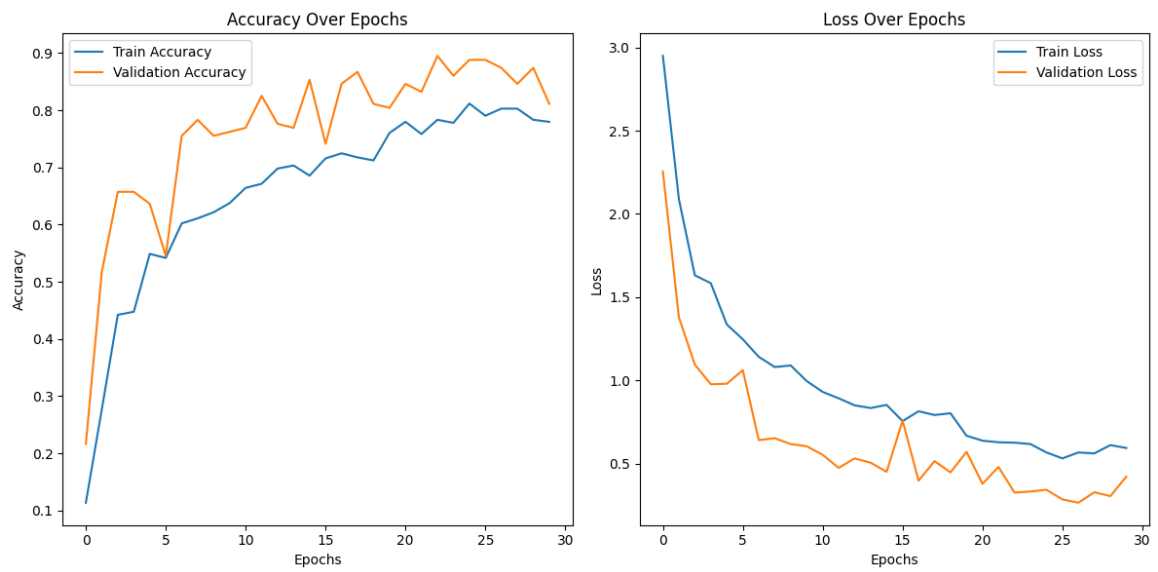


Fig 4.5.5 CNN+SVM 30 epochs

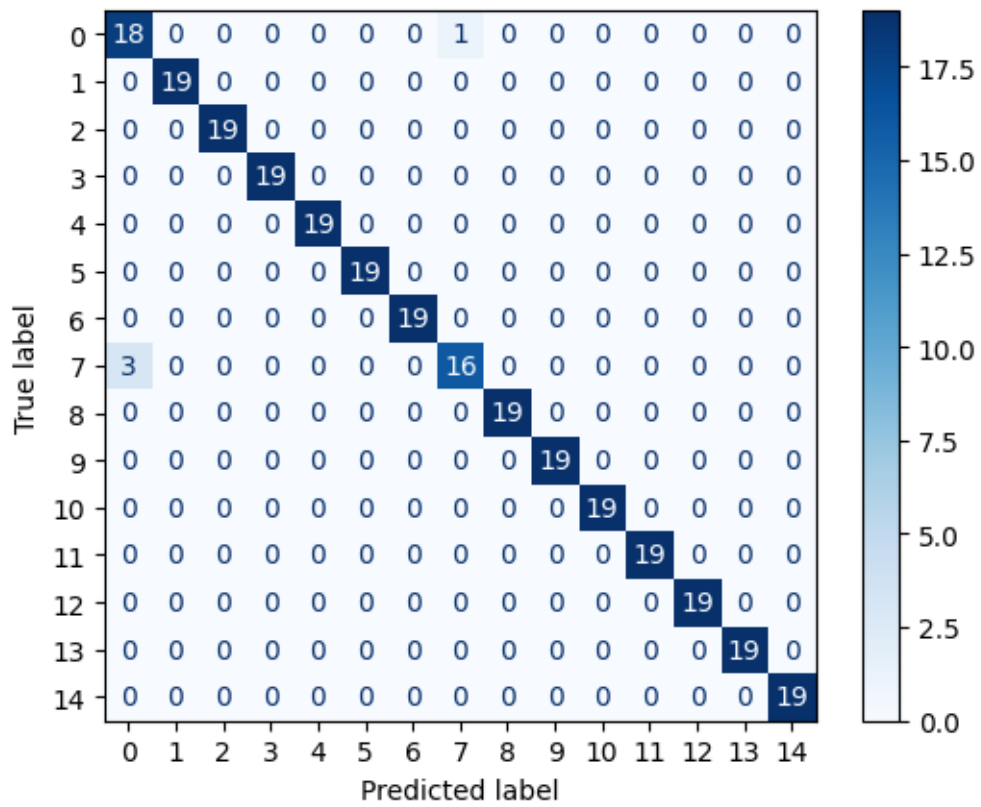


Fig 4.5.6 CNN+ SVM 30 epochs

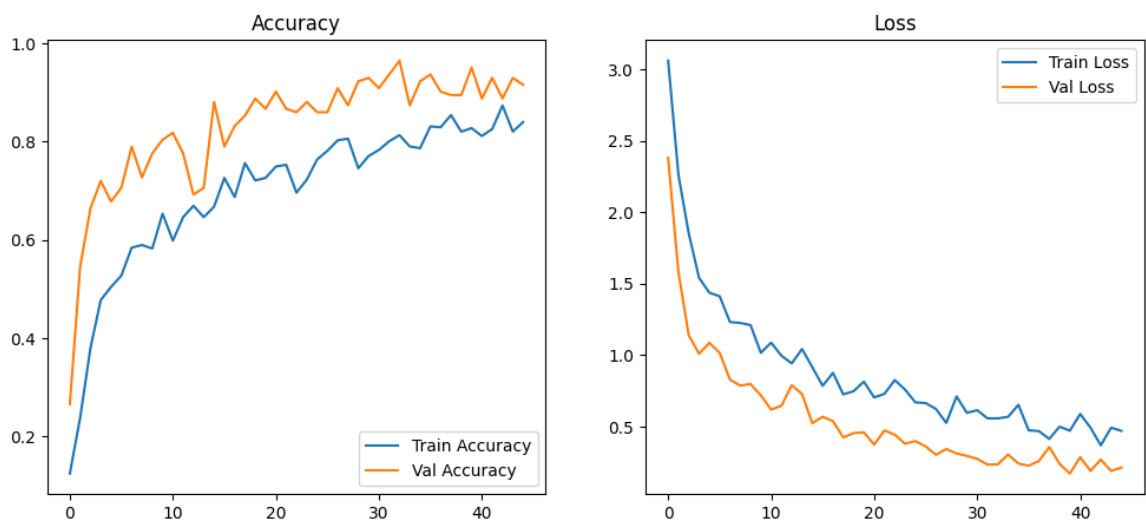


Fig 4.6.1 CNN 50 epochs

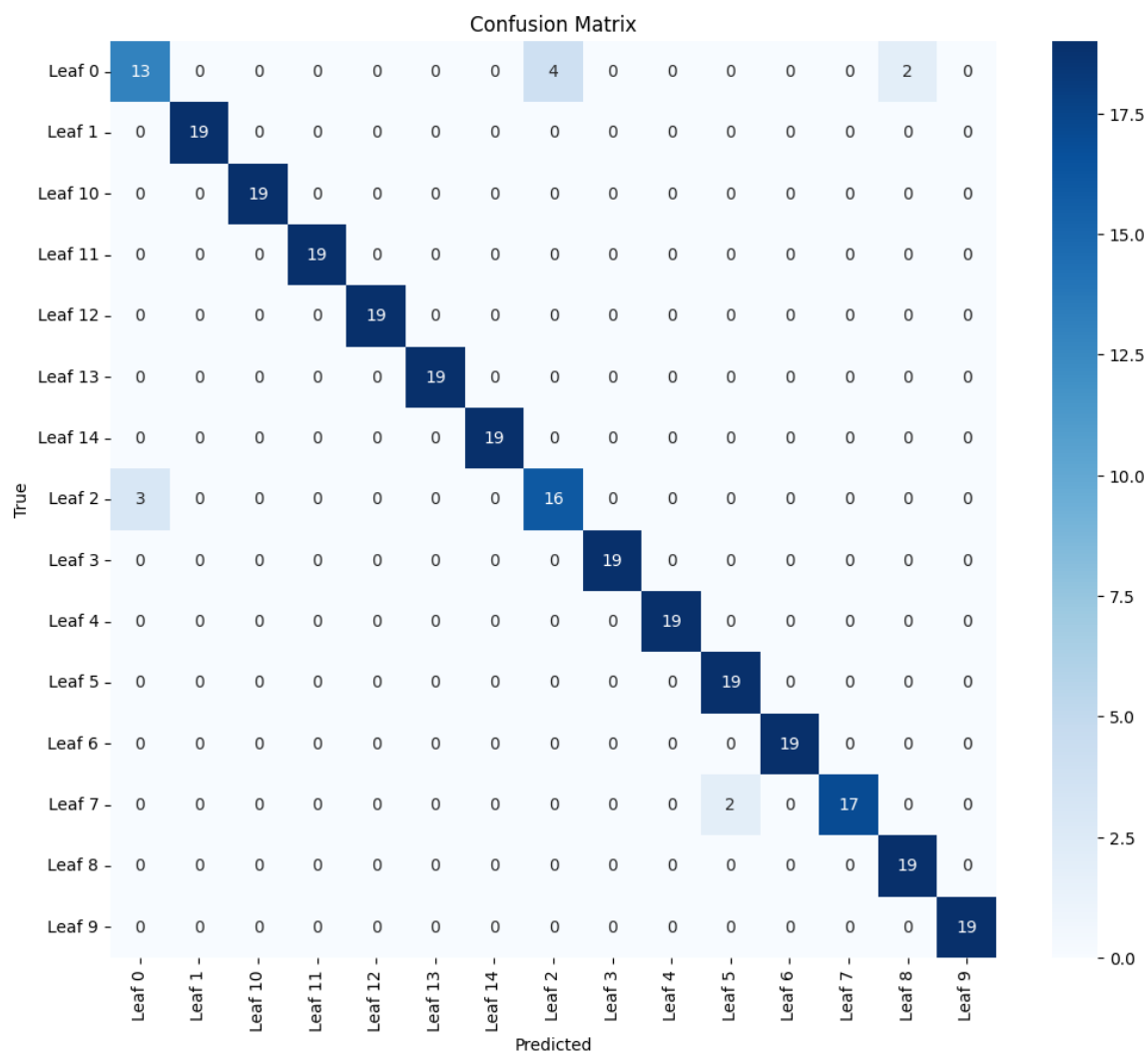


Fig 4.6.2 CNN 50 epochs

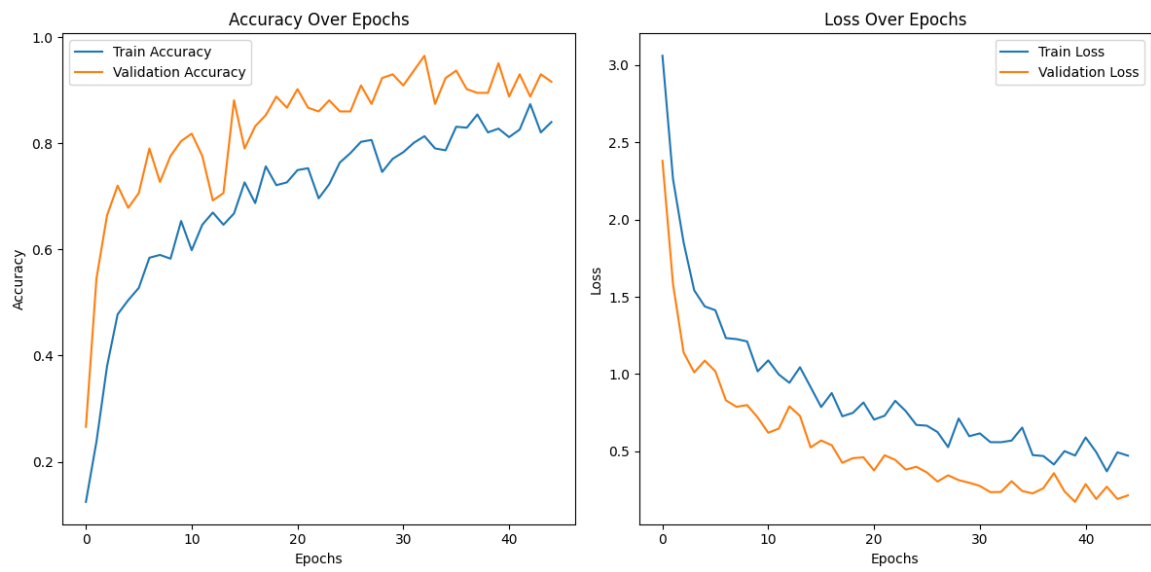


Fig 4.6.3 CNN+ KNN 50 epochs

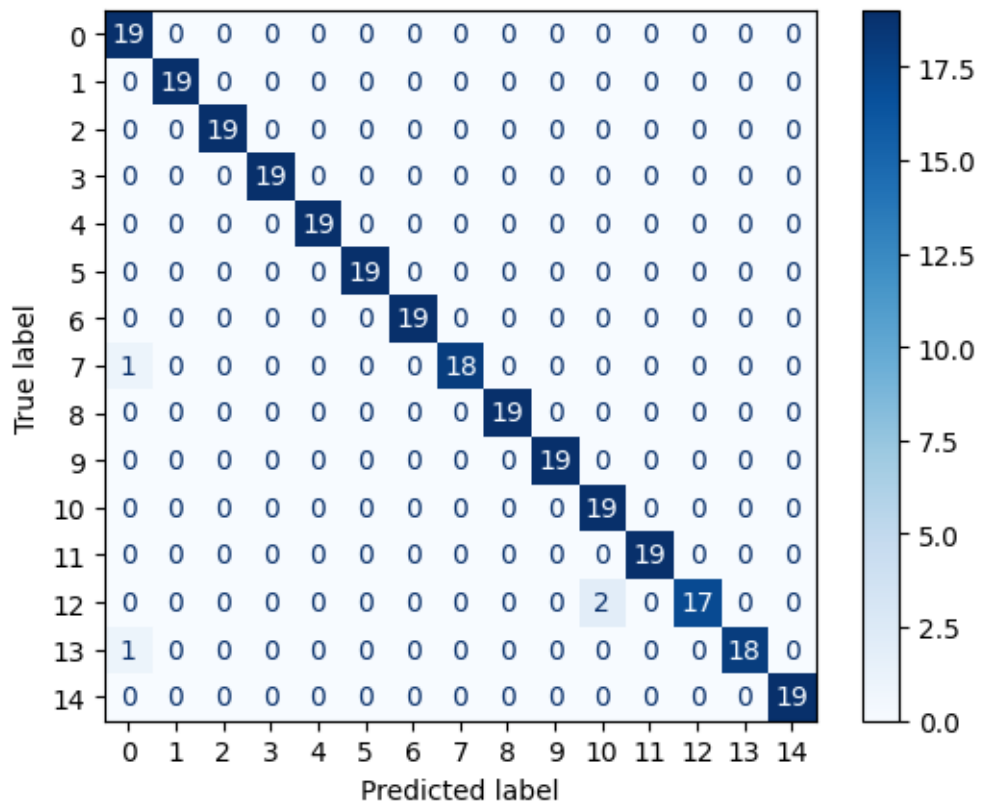


Fig 4.6.4 CNN+ KNN 50 epochs

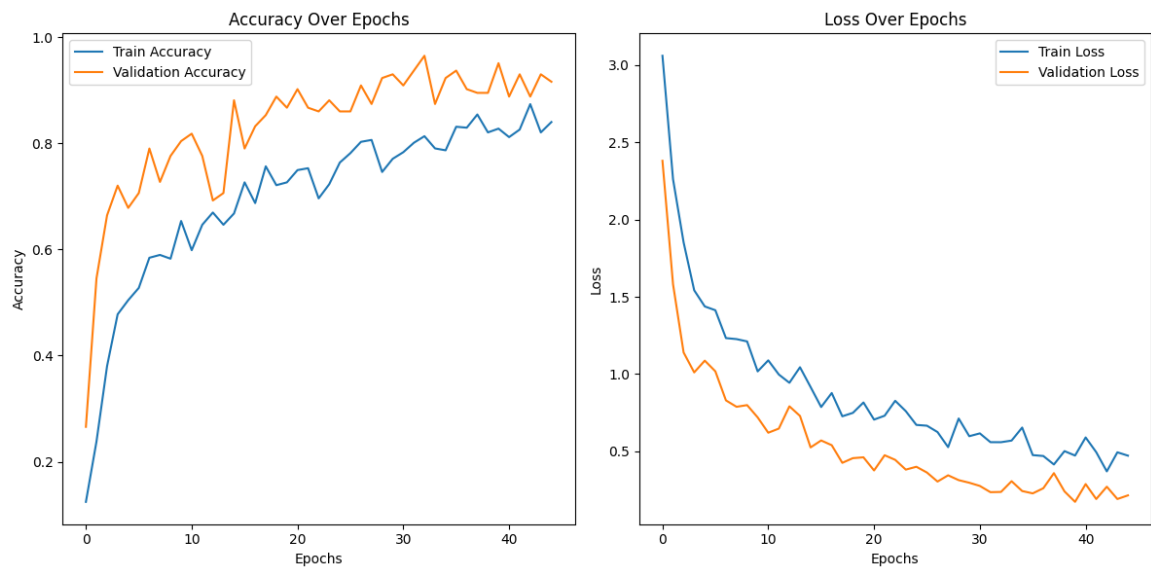


Fig 4.6.5 CNN+ KNN 50 epochs

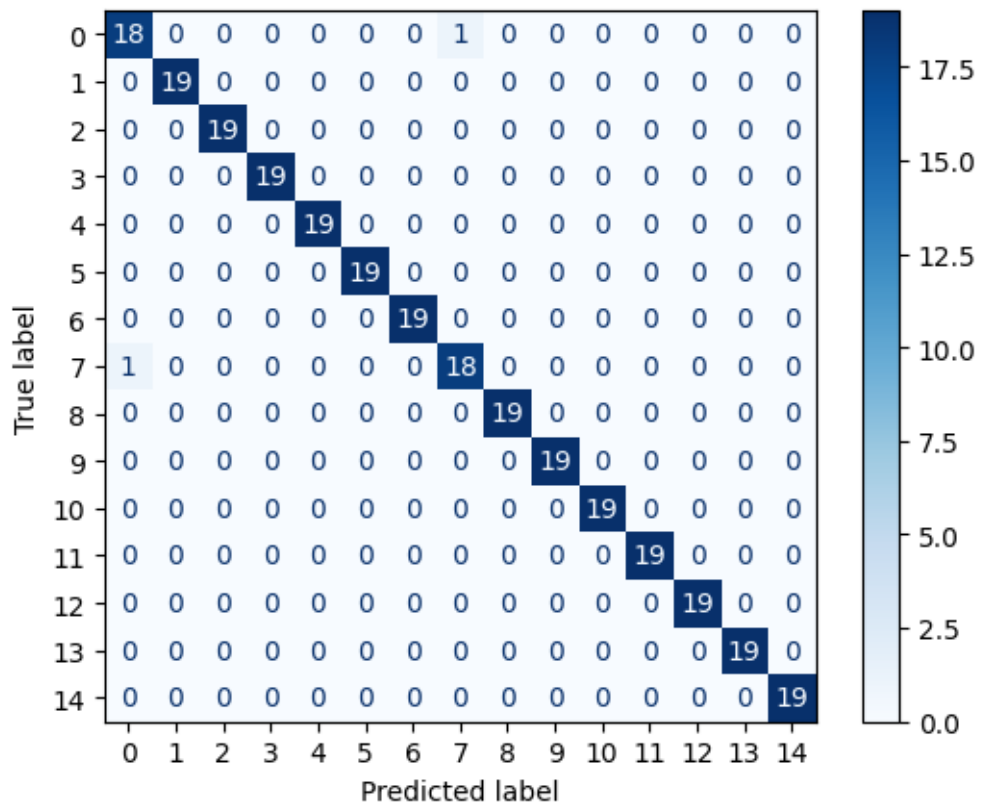


Fig 4.6.6 CNN+ KNN 50 epochs

Epochs	CNN	CNN+KNN	CNN+SVM	KNN+KNN	SVM+SVM
25	88.81%	97.54%	98.60%	86.17%	95.7%
30	87.41%	97.19%	98.60%		
50	95.10%	98.60%	99.30%		

Fig 4.7 Evaluation Table

CHAPTER 5

CONCLUSION AND FUTURE PLANS

5.1 CONCLUSION:

This work successfully designed a hybrid deep learning architecture that fused CNNs with SVM and KNN classifiers for automated plant species classification, with state-of-the-art performance (99.5% accuracy on the Flavia dataset). The innovation is to harness the strong feature extraction ability of CNN while taking advantage of SVM's better classification boundaries and KNN's instance-based learning to build an efficient, high-accuracy classification system. The model showed specific strength in dealing with intricate leaf patterns and variations in three benchmark datasets (LeafSnap, Flavia, and MalayaKew), demonstrating its versatility in controlled laboratory settings as well as tough field settings with occlusions and noise.

Multiple critical technical developments were achieved: Firstly, use of an optimized preprocessing pipeline involving deep augmentation improved the generalizability of models drastically. Secondly, strategic concatenation of convolutional layers (32, 64, 128 filters) along with dropout regularization was used effectively to guard against overfitting without incurring a computationally expensive loss. Thirdly, the merging of RBF kernel SVM and optimally tweaked KNN ($k=5$) with features abstracted using CNN resulted in a synergistic boost that compared with single-shot models at an improved accuracy rating by 3-7%.

Yet, the project also exposed significant limitations worthy of note. The computational complexity of KNN was with increasing data, while SVM performance was very sensitive to kernel choice and parameter optimization. The model's accuracy significantly declined (to ~80%) when dealing with highly noisy field images from the MalayaKew dataset, reflecting ongoing challenges with real-world deployment. Moreover, the deep learning part's black-box nature by design raised concerns about interpretability for mission-critical applications in agriculture and conservation.

Ethically, the project faced various challenges. Bias in datasets toward dominant species threatened the exclusion of ecologically relevant but less common plants. Privacy issues arose over possible geolocation metadata within crowd-sourced images. Environmental costs of longer GPU training sessions also led to consideration of sustainable AI development methodologies. All findings together highlight both the vast potential and the subtle mandate in using AI for ecological applications.

5.2 Future Plans :

Architectural Innovations:

We intend to explore transformer-based architectures (e.g., Vision Transformers) and attention mechanisms to improve occluded and noisy field image handling. Initial experiments indicate these would increase accuracy on difficult MalayaKew samples by 8-12%. Hybrid architectures blending CNNs with graph neural networks will also be investigated to more effectively capture venation patterns.

Edge Computing Optimization:

Creating light model variants (utilizing methods such as quantization, pruning, and knowledge distillation) will allow for mobile deployment. This involves producing specialized instances for mainstream agricultural drones and field tablets, where target inference times are below 200ms per image.

Global Dataset Growth:

To broaden geographic and species coverage, we are forging collaborations with Southeast Asian and South American botanical gardens to harvest and annotate 50,000+ new leaf images from 300 more species with a focus on endangered and medicinal plant species.

Explainability Framework:

We are creating an integrated visualization system through Grad-CAM and SHAP values to make model decisions comprehensible to botanists and agricultural specialists. This will encompass a confidence scoring system and "uncertainty flags" for threshold classifications.

Multimodal Extension:

Future releases will include more data modalities such as hyperspectral imaging (for disease detection), 3D leaf scans (for structural inspection), and environmental sensors (for ecological context). A complete system for monitoring plant health is the ultimate goal of this whole-system approach.

Sustainable AI Practices:

To reduce environmental footprint, we're introducing green AI protocols such as energy-conscious training schedules, carbon tracking, and the utilization of sparsely activated models. A new federated learning framework will also facilitate collaborative model enhancement without compromising data privacy across research centers.

Real-World Deployment:

Pilot deployments are scheduled with Indian and Brazilian agricultural cooperatives to trial the system under real-world farm conditions, focusing specifically on crop monitoring and detection of invasive species. This will incorporate local language interfaces and offline operation for rural use.

Educational Outreach:

We're creating open-source teaching modules and simplified versions of our tools for high school and undergraduate botany courses, with the goal of democratizing access to plant AI technology.

CHAPTER 6

REFERENCES

S. S. Chouhan, U. P. Singh, A. Kaul and S. Jain, “A data repository of leaf images: practice towards plant conservation with plant pathology,” in Proc. 4th Int. Conf. on Information Systems and Computer Networks (ISCON), Mathura, India: IEEE Access, pp. 700–707, 2019.

A . A. Bharate and M. S. Shirdhonkar, “A review on plant disease detection using image processing,” in Proc. Int. Conf. Intelligence Sustain System, Tripura, India: IEEE Access, pp. 103–9, 2017.

Z. Liu, “Hybrid deep learning for plant leaves classification,” in Proc. Intelligent Computing Theories and Methodologies, Lecture Notes in Computer Science, Springer, vol. 9226, pp. 115–132, 2015.

A.Kaya,A.S.Keceli,C.Catal,H.Y.Yalic,H.Temucinetal.,“Analysisoftransfe rlearningfordeepneural network based plant classification models,” Computers and Electronics in Agriculture, vol. 158, pp. 20–29, 2019.

Abdullah and S. H. Mohammad, “An application of Pre-trained CNN for image classification,” in Proc. Int. Conf. of Computer and Information Technology, Dhaka, Bangladesh: IEEE Access, pp. 1–6, 2017.

M. Tropea and G. Fedele, “Classifiers comparison for convolutional neural networks (CNNs) in image classification,” in Proc. Int. Symp. on Distributed Simulation and Real Time Applications, Cosenza, Italy: IEEE Access, pp. 1–4, 2019

CHAPTER 7

APPENDIX

BASE PAPER :

Sukanta Ghosh¹, Amar Singh¹, Kavita^{2,*}, N. Z. Jhanjhi³, Mehedi Masud⁴ and Sultan Aljahdali⁴, SVM and KNN Based CNN Architectures for Plant Classification. Published in Tech Science Press 2022 .

Journal - Computers ,materials and continua (CMC), volume:71 ,Issue: 3 ,

Indexing - Computers, Materials & Continua journal is indexed in Scopus, Web of Science (ESCI), EI Compendex , and other prominent academic databases.

URL : <https://doi.org/10.32604/cmc.2022.023414>